

Python Files And SQLite

Today we move from text inspection to persistent structured data.

Today

- read a CSV with Python
- create a SQLite database
- load ticket records
- query by category
- find high priority open tickets

Why it matters

- apps need stored data
- databases make queries safer
- SQLite requires no server
- Python includes sqlite3
- persistence survives restart

Same Data, More Structure

Bash gives quick answers. Python and SQLite give repeatable application logic.

Bash was good for

- quick search
- line counts
- simple category counts
- ad hoc reports
- working directly in terminal

Python plus SQLite is good for

- structured loading
- clean functions
- database storage
- more precise queries
- building toward an app

Memory, Files, Databases

Different storage choices fit different jobs.

Simple storage

- memory list: easy, disappears
- text file: easy, less structure
- CSV: shareable table format

Database storage

- SQLite: one local file
- PostgreSQL: production server
- MySQL: production server
- DuckDB: analytics-friendly local database

CSV Is Text That Looks Like A Table

Python can read CSV files using a built-in module.

```
id,student_name,category,priority,status
1,Ari,password,normal,open
2,Blake,network,high,open
```

```
import csv
reader = csv.DictReader(file)
```

DictReader uses the header row as field names.

Tables, Rows, Columns, IDs

These words show up in almost every database conversation.

Database words

- table: related records
- row: one record
- column: one field
- primary key: unique ID
- query: request for data

Ticket example

- table: tickets
- row: one helpdesk ticket
- column: priority
- ID: ticket number
- query: open high tickets

SQLite Stores A Database In One File

For beginner apps, this avoids server setup while still teaching database thinking.

tickets.csv

- > Python loader script
- > tickets.db
 - > SQL queries
 - > report output

The database file can be listed with `ls -l`.

Create The Tickets Table

A schema describes the shape of the data.

```
CREATE TABLE IF NOT EXISTS tickets (  
  id INTEGER PRIMARY KEY,  
  student_name TEXT NOT NULL,  
  category TEXT NOT NULL,  
  priority TEXT NOT NULL,  
  status TEXT NOT NULL,  
  issue TEXT NOT NULL  
);
```

NOT NULL means the field is required.

Load CSV Rows Into SQLite

Python reads each CSV row and inserts it into the table.

```
with CSV_PATH.open(newline="") as file:
    reader = csv.DictReader(file)
    for row in reader:
        connection.execute(
            "INSERT INTO tickets VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
            (...)
        )
    connection.commit()
```

Question marks are placeholders for safe values.

Ask The Database Questions

SQL can summarize and filter records.

```
SELECT category, COUNT(*) AS total  
FROM tickets  
GROUP BY category  
ORDER BY total DESC;
```

```
SELECT id, student_name, category, issue  
FROM tickets  
WHERE status = 'open' AND priority = 'high';
```

GROUP BY summarizes. WHERE filters.

The Database Survives Program Restart

This is the difference between practice code and app behavior.

Memory

- fast
- easy for demos
- gone after restart
- not enough for real tickets

SQLite file

- saved on disk
- can be reopened
- can be queried later
- first step toward real app storage

Default Plan: Live Code In Small Chunks

The best version of class is still a short live demo followed by student practice.

Live code if possible

- inspect tickets.csv
- run the loader
- show tickets.db appears
- run the analyzer
- explain output

Keep it controlled

- do not type every line from scratch
- use example files when needed
- pause after each chunk
- ask what changed
- then release students to lab

If Live Coding Fails, Use The Slides

This is the fallback, not the preferred plan.

Instructor move

- show the workflow diagram
- read code one chunk at a time
- explain what each chunk does
- show expected output
- then let students run or read the lab

Student move

- open LAB.md
- copy code carefully
- compare to slides
- run examples if possible
- answer reflection from the workflow

The goal is understanding the data flow, not watching perfect live typing.

What The Lab Does

There are two Python scripts and one database file.

1. tickets.csv
source data: 10 helpdesk tickets
2. load_tickets.py
creates tickets.db and loads CSV rows
3. analyze_tickets.py
asks questions of tickets.db
4. tickets.db
persistent SQLite database file

If students know this map, the lab is much less scary.

The Commands Students Run

These are the terminal steps at a high level.

```
$ python3 --version
```

```
$ python3 load_tickets.py
```

```
$ ls -l
```

```
$ python3 analyze_tickets.py
```

Optional:

```
$ sqlite3 tickets.db
```

The hard part is code meaning, not the number of commands.

Chunk 1: Import Tools

The script starts by loading Python modules.

```
import csv
import sqlite3
from pathlib import Path
```

```
CSV_PATH = Path("tickets.csv")
DB_PATH = Path("tickets.db")
```

csv reads CSV files. sqlite3 talks to SQLite. Path handles filenames.

Chunk 2: Connect To The Database

Opening the connection creates the database file if needed.

```
connection = sqlite3.connect(DB_PATH)
```

Meaning:

- open tickets.db
- create it if it does not exist
- use this connection for SQL commands

The connection is Python's link to the database file.

Chunk 3: Create The Table

The table is the structure that will hold ticket rows.

```
CREATE TABLE IF NOT EXISTS tickets (  
  id INTEGER PRIMARY KEY,  
  student_name TEXT NOT NULL,  
  category TEXT NOT NULL,  
  priority TEXT NOT NULL,  
  status TEXT NOT NULL,  
  created_date TEXT NOT NULL,  
  issue TEXT NOT NULL  
)
```

This is the shape of the database table.

Chunk 4: Make The Lab Repeatable

The loader clears old rows before inserting fresh rows.

```
connection.execute("DELETE FROM tickets")
```

Why?

- students may run the script more than once
- without this line, rows duplicate
- this keeps the beginner lab predictable

In a real app, deleting data requires care.

Chunk 5: Read The CSV

DictReader uses the header row as field names.

```
with CSV_PATH.open(newline='') as file:
```

```
    reader = csv.DictReader(file)
```

```
    for row in reader:
```

```
        print(row["category"])
```

Example field names:

id, student_name, category, priority, status

`row['category']` is easier than remembering field 3.

Chunk 6: Insert Rows

Each CSV row becomes one database row.

```
INSERT INTO tickets (  
    id, student_name, category, priority, status, created_date, issue  
) VALUES (?, ?, ?, ?, ?, ?, ?)
```

The ? marks are placeholders.

The actual row values are passed separately.

Placeholders are safer than pasting values into SQL text.

Chunk 7: Save And Close

commit writes the changes to the database file.

```
connection.commit()  
connection.close()
```

commit = save database changes
close = close the database connection

If changes do not appear, check whether commit happened.

After Running The Loader

Students should see a database file.

```
$ python3 load_tickets.py  
Loaded ticket data into tickets.db
```

```
$ ls -l  
... tickets.csv  
... tickets.db  
... load_tickets.py
```

tickets.db is the key artifact from the first script.

Chunk 1: Open The Database

The analyzer reads from tickets.db.

```
import sqlite3
```

```
connection = sqlite3.connect("tickets.db")  
connection.row_factory = sqlite3.Row
```

```
row_factory lets us use:  
row["category"]
```

The analyzer must point to the same database file the loader created.

Chunk 2: Count Tickets By Category

This SQL query replaces the Bash category-count pipeline.

```
SELECT category, COUNT(*) AS total
FROM tickets
GROUP BY category
ORDER BY total DESC
```

Plain English:

Group tickets by category, count each group, largest first.

This is the SQL version of `cut | sort | uniq -c | sort -nr`.

Same Question, Different Tool

Students already saw this idea in Bash.

Bash

```
tail -n +2 tickets.csv  
| cut -d, -f3  
| sort  
| uniq -c  
| sort -nr
```

SQL

```
SELECT category, COUNT(*)  
FROM tickets  
GROUP BY category  
ORDER BY COUNT(*) DESC
```

Chunk 3: Filter With WHERE

WHERE returns only rows that match the condition.

```
SELECT id, student_name, category, issue  
FROM tickets  
WHERE status = 'open' AND priority = 'high'  
ORDER BY id
```

Plain English:

Only open tickets that are also high priority.

AND means both conditions must be true.

After Running The Analyzer

This is what students should roughly see.

Tickets by category:

- network: 3
- software: 2
- password: 2
- access: 2
- hardware: 1

Open high priority tickets:

- #2 Blake (network): wifi drops during class
- #4 Drew (hardware): laptop keyboard not working
- #7 Gray (password): account locked after attempts
- #10 Jules (network): DNS lookup failing

If output is empty, rerun the loader first.

Part 1: Setup

The setup makes sure Python and files are ready.

```
$ mkdir -p ~/bootcamp/class03
```

```
$ cd ~/bootcamp/class03
```

```
$ python3 --version
```

```
$ ls
```

```
$ head tickets.csv
```

Most setup problems are file-location problems.

Part 2: Loader Script

The loader creates and fills the database.

Students create

- load_tickets.py
- imports
- database connection
- table creation
- CSV reading
- INSERT statements
- commit and close

Students verify

- run `python3 load_tickets.py`
- look for tickets.db
- answer what file appeared
- explain why it matters

Part 3: Analyzer Script

The analyzer asks the database questions.

Questions answered

- Which category is most common?
- How many open high-priority tickets exist?
- Which students have those tickets?

SQL tools used

- SELECT reads rows
- COUNT counts rows
- GROUP BY groups rows
- WHERE filters rows
- ORDER BY sorts results

Part 4: Optional sqlite3

The command-line database tool is optional.

```
$ sqlite3 tickets.db
sqlite> .tables
sqlite> SELECT COUNT(*) FROM tickets;
sqlite> SELECT category, COUNT(*) FROM tickets GROUP BY category;
sqlite> .quit
```

If sqlite3 is not installed, skip it. Python output is enough.

If The Lab Breaks

Use these checks before changing code randomly.

Terminal checks

- pwd
- ls
- head tickets.csv
- python3 --version
- ls -l tickets.db

Python checks

- read file name in traceback
- read line number
- check indentation
- check CSV header
- rerun loader before analyzer

Most Common Error: File Not Found

This usually means the CSV is not where the script expects.

Error:

`FileNotFoundError: tickets.csv`

Check:

```
$ pwd
```

```
$ ls
```

```
$ head tickets.csv
```

Fix:

Put tickets.csv in the current folder or fix CSV_PATH.

Path problems are normal beginner problems.

Most Common Error: Indentation

Python uses indentation as part of the language.

Error:
IndentationError

Likely cause:

- missing spaces inside for loop
- copied code shifted left or right

Fix:
Open the file and check the line number.

Spaces at the start of Python lines matter.

Add A Status Count

This is the easiest useful extension.

```
SELECT status, COUNT(*) AS total  
FROM tickets  
GROUP BY status  
ORDER BY total DESC
```

Expected:

open: 7

closed: 3

Same pattern as category count, different column.

What Students Should Remember

They do not need to memorize every line of code.

tickets.csv

simple text source data

load_tickets.py

creates tickets.db and inserts rows

tickets.db

persistent SQLite database file

analyze_tickets.py

runs SQL queries and prints a report

This is the whole lab in one slide.

Debug Python And SQLite Calmly

Most problems are path, filename, indentation, or schema issues.

Likely errors

- tickets.csv not found
- tickets.db in wrong folder
- IndentationError
- KeyError from CSV header
- SQL typo
- forgot connection.commit

Debug checks

- pwd
- ls -l
- head tickets.csv
- read traceback file and line
- check column names
- rerun loader

Query Without Writing More Python

If the sqlite3 command exists, students can inspect the database directly.

```
$ sqlite3 tickets.db
sqlite> .tables
sqlite> SELECT COUNT(*) FROM tickets;
sqlite> SELECT category, COUNT(*) FROM tickets GROUP BY category;
sqlite> .quit
```

If sqlite3 is missing, Python query output is enough.

This Is The Data Layer Of Apps

Almost every useful application needs some version of this idea.

Examples

- helpdesk systems
- inventory apps
- dashboards
- QA tracking
- security findings
- AI tool logs

Transferable concepts

- records
- IDs
- schema
- queries
- persistence
- error handling

Ungraded Skill Practice

Students save scripts, output, and a short explanation for their own notes.

Practice evidence

- python version
- ls -l showing tickets.db
- load_tickets.py
- analyze_tickets.py
- query output

Writing

- what file stores the database
- most common category
- open high tickets
- CSV vs database
- why apps need persistence

What Success Looks Like Today

Students can move data from a file into a persistent database.

Students should be able to say

- I can read a CSV with Python
- I can create a SQLite database
- I can load rows into a table
- I can run a query
- I can explain persistence

Next class

- FastAPI
- HTTP GET and POST
- JSON
- automatic docs
- debugging an app